Autonomous ETL Pipelines for Salesforce M&A Consolidation: A Reference Framework, Jitterbit Mapping and Reproducible Simulation Study

Imran Basha Shaik*, Sahla Sultana, Khader Moinuddin Shaik

Imran Basha Shaik, Frisco, TX, USA · ORCID 0009-0002-1210-5646 · [corresponding author e-mail]

Sahla Sultana, Frisco, TX, USA

Khader Moinuddin Shaik, Frisco, TX, USA

*Corresponding author

Abstract

Merging companies that each run Salesforce must consolidate their customer data into a single organization while both source systems remain in use. This requires reconciling different schemas and picklists, resolving duplicate customers across and within organizations, preserving relationships between records, and keeping the target current until cutover. This article proposes a reference framework for an autonomous extract-transform-load pipeline for this task. It combines partitioned bulk extraction, declarative mapping templates, rule-based and statistical quality gates, master data management with deterministic and blocked fuzzy matching, constrained loading with per-record error capture, and change data capture guarded by a data-health circuit breaker. Each stage is mapped to Jitterbit Integration Studio constructs. The framework was implemented as an open reference prototype and evaluated on synthetic two-organization scenarios with known ground truth at three scales, from 32,040 to 383,973 source records, over five random seeds. At a preset threshold, hybrid matching reached pairwise F1 scores of 0.918 for accounts and 0.885 for contacts; calibrating the threshold per object would raise these to 0.938 and 0.963. The quality gates prevented all load failures and all silent loss of unmapped picklist values without wrongly flagging any clean record, whereas removing them produced about 1,000 load failures per run. Over five simulated days of hourly incremental synchronization, replayed events were ignored, an injected upstream fault was contained in the batch in which it occurred, and the target agreed with a full rebuild on more than 99.3% of field values. Validation on a production deployment remains future work.

Keywords: Data integration; Entity resolution; Master data management; Change data capture; Data quality; Extract-transform-load; Salesforce

1. Introduction

Mergers and acquisitions (M&A) frequently leave the combined company with two or more customer relationship management (CRM) environments that describe overlapping customers in different ways. When the environments are Salesforce organisations ("orgs"), consolidation into a single target org requires the integration team to reconcile structural differences in objects and fields, translate picklist vocabularies, normalise date, time-zone and contact formats, identify records that describe the same account or person, and re-parent child records so that relationships survive the move. The work must usually be completed while both source orgs remain in daily use, which means the target has to be kept current between the initial bulk load and the final cutover.

Traditional extract-transform-load (ETL) practice was shaped by data warehousing, where sources are comparatively stable and load windows can be scheduled well in advance (Kimball & Caserta, 2004; Vassiliadis, 2009). Consolidation projects stress that model in several ways. Data volumes and the variety of formats have grown (Saggi & Jain, 2018; Singh & Reddy, 2015), duplicate detection is a

research field in its own right (Christen, 2012a; Elmagarmid et al., 2007), and data quality failures that are caught late are expensive to repair (Batini et al., 2009). Integration Platform as a Service (iPaaS) products such as Jitterbit Harmony provide connectors, mapping tools and schedulers that reduce hand-written code, but they do not by themselves specify how matching, quality control, lineage and incremental synchronisation should be combined for a CRM merger.

The academic literature offers mature building blocks for each of these concerns, yet few studies assemble them into an end-to-end design for CRM consolidation and evaluate the assembled design against known answers. Evaluations of consolidation tooling are also hard to reproduce, because merger data are confidential. This paper responds to both problems. Following a design science approach (Hevner et al., 2004; Peffers et al., 2007), it makes four contributions:

- A reference framework for autonomous ETL in Salesforce consolidation that combines partitioned extraction, declarative mapping templates, quality gates, master data management (MDM), constrained loading, change data capture (CDC) and data-health monitoring, with explicit control points and audit artefacts (Section 3).
- A mapping of each framework stage onto Jitterbit Integration Studio constructs, so that the design can be implemented on that platform (Section 3.8).
- An open, reproducible simulation that generates two Salesforce-like orgs with differing schemas, known duplicates, deliberately difficult near-duplicates and injected defects, together with ground truth that the pipeline never sees (Section 4).
- An empirical characterisation of the framework's behaviour, including matching accuracy and its sensitivity to thresholds, the contribution of each quality gate, scaling behaviour, and the agreement between incremental synchronisation and a full rebuild (Section 5).

Throughout, we distinguish between what the simulation demonstrates and what would require a production deployment to establish. Section 6 discusses the findings and Section 7 sets out the threats to validity.

2. Related Work

2.1 ETL architectures and data integration

ETL processes extract data from heterogeneous sources, transform them into a common representation and load them into a target. Vassiliadis (2009) surveys ETL research across conceptual modelling, the individual extraction, transformation and loading stages, and end-to-end concerns such as optimisation and resumption. Simitsis et al. (2005) treat ETL workflows as state spaces that can be optimised by reordering activities, which supports separating transformation logic from its execution plan. Practitioner texts emphasise staging areas, conformed dimensions and the need for repeatable, auditable jobs (Kimball & Caserta, 2004). Schema matching, the identification of corresponding elements in different schemas, is the precursor to any mapping between two orgs (Rahm & Bernstein, 2001).

Big data research extends these ideas to larger and faster-changing sources. Surveys of analytics platforms and distributed frameworks document trade-offs among throughput, latency and fault tolerance and show why monolithic, single-node jobs struggle as volumes grow (Inoubli et al., 2018; Singh & Reddy, 2015). Cloud architectures add elastic provisioning, which suits the bursty workload of a migration (Khan et al., 2013). Saggi and Jain (2018) and Alharthi et al. (2017) describe how volume, velocity and variety, together with organisational barriers, complicate integration projects. Dong and Srivastava (2015) frame these challenges specifically for data integration, identifying schema alignment, record linkage and data fusion as its core tasks. These are the same three tasks a CRM consolidation must perform.

2.2 Entity resolution and master data management

Entity resolution (also called record linkage or deduplication) identifies records that refer to the same real-world entity. The probabilistic model of Fellegi and Sunter (1969) remains the foundation of weighted matching, and Winkler (1990) introduced the string comparator widely used for names. Elmagarmid et al. (2007) survey duplicate detection techniques, Christen (2012a) gives a comprehensive treatment of data matching, and Köpcke and Rahm (2010) compare matching frameworks. Because comparing every pair of records grows quadratically, practical systems use blocking or indexing to restrict comparisons to plausible candidates (Christen, 2012b; Papadakis et al., 2020). Christophides et al. (2020) review end-to-end entity resolution workflows, including incremental settings in which records arrive over time. Gruenheid et al. (2014) show that incremental linkage can diverge from batch linkage unless existing clusters are revisited when new evidence arrives, a point that bears directly on CDC-driven consolidation.

Master data management builds on entity resolution by maintaining a single authoritative ("golden") record per entity, together with survivorship rules that decide which source supplies each attribute, and a cross-reference from source identifiers to golden identifiers (Loshin, 2009). In an M&A consolidation, that cross-reference is also the lineage record that allows any target value to be traced back to its source.

2.3 Data quality and automated validation

Data quality is multidimensional, spanning accuracy, completeness, consistency and timeliness (Wang & Strong, 1996). Rahm and Do (2000) classify data cleaning problems as single-source or multi-source and as schema-level or instance-level, a taxonomy that maps directly onto the defects found when two CRM orgs are merged. Batini et al. (2009) compare methodologies for assessing and improving quality. More recent work treats validation as code: Schelter et al. (2018) describe declarative "unit tests for data" that run at scale, and Breck et al. (2019) describe schema-based validation and drift detection in production pipelines. Roh et al. (2021) review data collection and preparation, stressing that quality controls should sit early in a pipeline rather than after data have been consumed.

2.4 Change data capture and incremental maintenance

Change data capture propagates inserts, updates and deletes from a source instead of repeating full extracts. Kleppmann (2017) discusses log-based CDC, idempotent consumers and the at-least-once delivery semantics that make duplicate events a normal condition rather than an exception. Salesforce exposes change events for supported standard objects and all custom objects through its Change Data Capture feature (Salesforce, 2026a), and its Bulk application programming interface (API) 2.0 supports asynchronous ingest and query of large datasets (Salesforce, 2026b). An incremental consolidation pipeline must therefore handle replayed events, maintain checkpoints, and decide what to do with change events that would break integrity rules.

2.5 Research gap

The literature supplies the components of a consolidation pipeline: ETL architecture, schema matching, entity resolution with blocking, survivorship, declarative validation and CDC. What remains under-specified is how these components should be combined, controlled and audited for the particular case of merging Salesforce orgs, and how such a combined design behaves when its outputs can be checked against known answers. Industrial descriptions of iPaaS-based migrations rarely report matching accuracy, the effect of validation gates, or whether incremental synchronisation converges to the same state as a full reload. This study addresses that gap with an explicit framework and a reproducible evaluation in which every reported number can be regenerated.

3. Reference Framework

3.1 Design objectives and assumptions

The framework is designed to satisfy six objectives: (O1) every source record is either represented in the target or accounted for in an audit log; (O2) duplicate entities are merged with measurable precision and recall; (O3) no record that would violate target constraints reaches the load stage; (O4) every target value can be traced to its source records; (O5) the target is kept current between bulk load and cutover, and replayed change events do not alter the result; and (O6) faults in upstream data are detected and contained automatically. The design assumes API access to every source org and to the target, a staging area for intermediate data, and a single acquirer org whose values take precedence when two sources are equally recent.

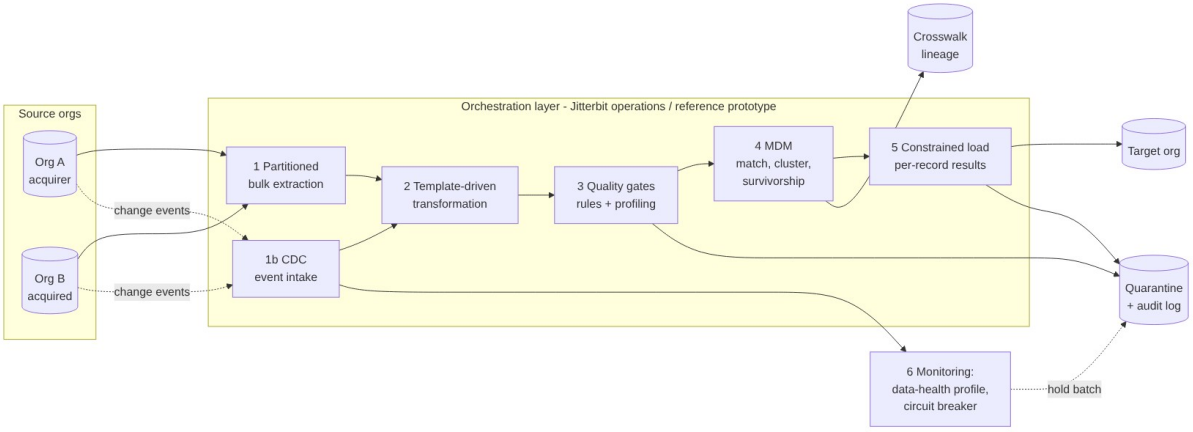


Figure 1. Reference architecture. Numbered stages run in the orchestration layer (Jitterbit operations in a production deployment; the Python reference prototype in this study). Rejected and held records go to quarantine with audit entries; the crosswalk records lineage from source to golden identifiers.

Figure 1 shows the architecture and Table 1 summarises the purpose, inputs, outputs and controls of each stage.

Table 1. Pipeline stages and their controls

Stage	Purpose	Main output	Controls
1 Extraction	Capture a baseline snapshot of each source object	Compressed staged shards partitioned by creation year	Partition keys; parallel workers; shard manifest
1b CDC intake	Capture creates, updates and deletes after the baseline	Ordered micro-batches of change events	Event identifiers; idempotent skip of replays
2 Transformation	Map each org's schema, picklists and formats to the canonical model	Canonical records with normalised match keys	Versioned templates; unmapped values flagged, never defaulted
3 Quality gates	Stop records that would break target rules; flag statistical outliers	Accepted records; quarantine with audit entries	Rule catalogue; outlier fence; hold of children whose parent is quarantined
4 MDM	Find duplicates and build golden records	Golden accounts and contacts; crosswalk	Deterministic rules; blocking; weighted score threshold; survivorship
5 Load	Write golden records and re-parented children	Target records; per-record error log	Batches of 10,000; row-level fallback on failure

Stage	Purpose	Main output	Controls
6 Monitoring	Watch data health of each CDC batch	Alerts; held batches	Profile baseline; z-score threshold; circuit breaker

3.2 Partitioned extraction

Each source object is partitioned on its creation date and the partitions are staged in parallel as compressed files with a manifest. Partitioning bounds the size of any single request, allows interrupted extractions to resume at the partition level, and mirrors the way bulk query jobs are typically split to stay within platform limits (Salesforce, 2026b). The manifest lists each partition, its object and its record count, so that the reconciliation step can confirm that nothing was lost before transformation began.

3.3 Template-driven transformation

Transformation logic is expressed as declarative mapping templates rather than code. A template names, for each source org and object, the source field that supplies each canonical field, the date and time format and time zone of the source, and the versioned picklist map to apply. Org B in the evaluation, for example, stores timestamps as US Central local time in a month-first format, labels the account name field Company_Name__c, and uses its own industry and stage vocabularies; its template converts all of these to the canonical model. Adding a further org requires a new template, not new code.

Two rules govern transformation. First, a value that cannot be mapped (an unknown picklist value or an unparseable date) is recorded as an issue on the record and passed to the quality gates; it is never silently replaced by a default. Second, the transformation stage computes normalised match keys used later by MDM: company names lower-cased with punctuation and legal suffixes removed, telephone numbers reduced to their last ten digits, web domains stripped of scheme and "www", e-mail addresses lower-cased and checked for syntactic validity, and first names mapped through a standard nickname table.

3.4 Quality gates

The quality gates apply a catalogue of rules, each recorded in the audit log with the rule name, the field, and the action taken. Blocking rules quarantine a record when a required field is missing, an e-mail address is syntactically invalid, an opportunity amount is negative, a transformation issue was recorded, or a lookup refers to an account that does not exist in any source. A child record whose parent account exists but was itself quarantined is held rather than rejected, so that it can be released once the parent is repaired. Non-blocking rules flag records for review without stopping them. In the prototype, one such rule computes a robust outlier fence on log-transformed annual revenue (upper quartile plus 1.5 times the interquartile range) to catch unit errors that no field-level rule would detect.

3.5 Master data management

MDM runs in two passes, accounts first and then contacts, because contact matching uses the golden account to which each contact belongs.

Matching combines deterministic and probabilistic evidence. Deterministic rules link accounts that share a normalised web domain or the same normalised name and telephone number, and link contacts that share a valid e-mail address. Probabilistic matching then compares candidate pairs produced by blocking. Accounts are blocked on the first four characters of the normalised name, the first five characters of its longest token, and the telephone number. Contacts are blocked within their golden account on the first two characters of the surname and of the first name, and globally on telephone number. Each candidate pair receives a weighted score in the spirit of Fellegi and Sunter (1969): Jaro-Winkler similarity on names (Winkler, 1990) and exact agreement on city, telephone, domain or e-mail,

with weights renormalised over the attributes that both records actually contain. Pairs at or above a threshold τ are linked, and clusters are formed as connected components of all links.

Survivorship builds one golden record per cluster: for each attribute it takes the most recently modified non-empty value, breaking ties in favour of the acquirer org, and it keeps the earliest creation date. Every source identifier is written to a crosswalk with its golden identifier, which provides the lineage required by objective O4. Opportunities are not deduplicated, because they represent distinct transactions, but their account references are rewritten to golden account identifiers.

3.6 Constrained load and reconciliation

The target schema enforces the constraints that the target org would enforce: required fields, valid lookups, restricted picklists, an e-mail format check and non-negative amounts. Records are loaded in batches of 10,000. If a batch fails, it is retried record by record and each failure is written to an error log, mirroring the per-record results returned by a bulk ingest job. After loading, a reconciliation step confirms that every source record is either represented in the target (directly or through the crosswalk) or present in the audit log, that no lookup in the target points to a missing account, and that the sum of opportunity amounts in the target equals the sum over accepted source opportunities.

3.7 Change data capture and monitoring

After the baseline load, change events are applied in hourly micro-batches. Each event carries an identifier, and any identifier already processed is skipped, which makes the consumer idempotent under at-least-once delivery (Kleppmann, 2017). Each event is transformed and validated with the same templates and rules as the bulk path. An update is applied to its source record and the affected golden record is recomputed by survivorship. A new contact is matched against the existing contacts of its golden account (and against the e-mail index) and either joins an existing golden record or creates a new one. Deletions remove the corresponding target record.

Before a batch is applied, the monitoring stage computes a data-health profile, in the prototype the share of contact updates with a blank e-mail address, and compares it with the mean and standard deviation of the first 24 batches. If the batch exceeds the baseline by more than four standard deviations, a circuit breaker holds the entire batch in quarantine for review instead of applying it. This protects the target from an upstream fault, such as a faulty synchronisation job that blanks fields, that would pass record-level validation because blank e-mail addresses are individually legal.

3.8 Mapping to Jitterbit Integration Studio

The framework is platform-neutral, but it was designed to be implemented with Jitterbit Harmony. Table 2 maps each stage to Integration Studio constructs. This mapping is a design proposal: the evaluation in this paper exercises the reference prototype, not a Jitterbit runtime, and specific connector capabilities should be confirmed against the current platform documentation (Jitterbit, 2026).

Table 2. Mapping of framework stages to Jitterbit Integration Studio constructs (design mapping, not evaluated)

Framework stage	Jitterbit construct	Notes
Extraction	Salesforce connector query or bulk query activities, one operation per object and partition	Partition by CreatedDate or SystemModstamp to bound job size
CDC intake	Scheduled operation with incremental query on SystemModstamp, or an external subscriber to Salesforce change events feeding an API endpoint	Persist the last checkpoint and processed event identifiers in project variables or a staging table

Framework stage	Jitterbit construct	Notes
Transformation	Transformations (mappings) with Jitterbit Script; picklist maps held in lookup tables	Version templates with the project
Quality gates	Conditional logic in scripts; failed records routed to a quarantine target	Write rule, field and action to an audit table
MDM	Script or external matching service called from an operation; crosswalk written to a staging database	Matching at scale may be better delegated to a database or service
Load	Salesforce connector bulk upsert activities using external identifiers	Capture per-record success and failure files
Monitoring	Operation schedules, operation actions on failure, and operation logs and alerts in the management console	Profile check runs as the first step of each CDC operation

4. Evaluation Design

4.1 Research questions

The evaluation addresses five research questions (RQs). RQ1: how accurately does the hybrid matcher merge duplicate accounts and contacts, and how much do its deterministic and probabilistic parts each contribute? RQ2: how sensitive is matching to the threshold τ, and what does blocking cost in accuracy? RQ3: which defects do the quality gates catch, which do they miss, and what happens without them? RQ4: how does run time grow with data volume, and which stage dominates? RQ5: does incremental synchronisation stay correct under replayed events and upstream faults, and does it converge to the same state as a full rebuild?

4.2 Synthetic scenario and ground truth

No real merger data were available for this study, and real CRM data would in any case carry privacy obligations and lack verified duplicate labels. We therefore built a generator that produces two Salesforce-like orgs with known ground truth. Org A (the acquirer) uses standard field names, ISO-8601 timestamps in Coordinated Universal Time (UTC) and the canonical picklist values. Org B (the acquired company) uses custom field names, US Central local timestamps in a month-first format, and its own vocabulary in which several values map to one canonical value (for example, "Tech" and "Hi-Tech" both map to Technology), plus a few values with no mapping.

The generator first creates canonical companies and people using the Faker library, then distributes them across orgs: 50% of companies appear only in Org A, 35% only in Org B and 15% in both. A further 4% of accounts and 3% of contacts receive a second record within the same org. Copies are perturbed in the ways that make matching difficult in practice: typographical errors; changed or added legal suffixes; "and" and "&" interchanged; upper-casing; nicknames and initials; personal instead of work e-mail addresses; missing, reformatted or different telephone numbers; and a different city for 20% of duplicate accounts. Two kinds of deliberately difficult non-duplicates are added: sibling companies that share a name stem (for example, a "Holdings" and a "Logistics" company with the same root name), half of which share the parent's web domain, and pairs of people with the same surname at the same account.

Defects are injected at low, realistic rates with their type recorded in the ground truth: invalid e-mail addresses (1.0% of contacts), missing required names (0.5% of contacts, 0.3% of accounts), references

to non-existent accounts (0.5% of contacts and opportunities), negative amounts (0.5% of opportunities), unmapped picklist values and unparseable dates in Org B (0.5% and 0.3%), revenue values inflated a thousand-fold (0.3% of accounts) and first and last names swapped (0.5% of contacts). The last two are included because they are plausible-looking and not covered by any field-level rule. The ground truth file is read only by the evaluation code.

4.3 Experiments and metrics

Table 3 lists the experiments. Matching is scored with pairwise precision, recall and F1 against the true entity labels of the records that entered MDM (Christen, 2012a). Quality gates are scored by the share of records carrying each injected defect that were flagged, and by false positives among records with no injected defect. Run time is wall-clock time for the whole batch pipeline. For CDC, events were generated at 2% of all records per day for five days, spread across hourly batches, with 1% of events re-delivered to simulate at-least-once delivery. In batch 83 (day 4, 10:00), every event was a contact update in which a simulated faulty upstream job had blanked the e-mail and telephone fields. After the CDC run, the full pipeline was re-run on the final state of the sources (excluding the held batch) and the two targets were compared.

Table 3. Experiments

ID	Question	Data	Repetitions
E1	RQ4 scaling	5,000, 20,000 and 60,000 canonical companies (32,040 to 383,973 source records)	3 runs per scale; median reported
E2	RQ1 matching and ablation	10,000 canonical companies (about 64,000 source records)	5 random seeds
E3	RQ2 threshold sweep and blocking	Same as E2 (sweep); 2,000 companies (blocking)	5 seeds (sweep); 1 (blocking)
E4	RQ3 quality gates, with and without	Same as E2	5 seeds
E5	RQ5 CDC, idempotency, fault containment, equivalence	20,000 companies (128,037 source records); 120 hourly batches	1 run

The reference prototype and the experiment scripts were written with the assistance of a generative AI coding tool (Claude, Anthropic); the authors reviewed the code and verified every reported number against the raw experiment output. All experiments ran on a Linux container with two CPU cores, Python 3.13, RapidFuzz 3.14 for string similarity and SQLite as the stand-in target. Absolute times therefore describe local computation and exclude network latency and Salesforce API limits.

5. Results

5.1 Matching accuracy (RQ1)

Table 4 reports pairwise matching quality at the pre-set threshold $\tau = 0.85$, averaged over five seeds. On average 11,873 account records and 27,975 contact records entered MDM per seed, containing 2,055 and 2,015 true duplicate pairs respectively.

Table 4. Pairwise matching quality at $\tau = 0.85$ (mean ± standard deviation over 5 seeds)

Object	Strategy	Precision	Recall	F1
Account	Deterministic only	0.902 ± 0.015	0.850 ± 0.011	0.875 ± 0.011

Object	Strategy	Precision	Recall	F1
Account	Fuzzy only	0.999 ± 0.001	0.769 ± 0.010	0.869 ± 0.007
Account	Hybrid	0.906 ± 0.015	0.931 ± 0.008	0.918 ± 0.011
Contact	Deterministic only	0.997 ± 0.003	0.694 ± 0.012	0.818 ± 0.009
Contact	Fuzzy only	0.999 ± 0.001	0.753 ± 0.004	0.859 ± 0.003
Contact	Hybrid	0.997 ± 0.003	0.796 ± 0.007	0.885 ± 0.004

The hybrid strategy gave the highest F1 for both objects because deterministic and probabilistic evidence find different duplicates. One result was unexpected. For accounts, almost all of the hybrid matcher's precision loss came from the deterministic rule, not from fuzzy matching: fuzzy matching alone was almost perfectly precise (0.999), whereas the shared-domain rule merged subsidiaries that use their parent's web domain. A rule that looks safe because it requires an exact match can therefore be the main source of false merges. For contacts, the remaining errors were almost all missed matches (recall 0.796). In the seed-7 run, every one of the 409 missed contact pairs lacked a shared e-mail address (one record used a personal address or none), and a third also lacked a telephone number on at least one side, leaving the names as the main evidence.

5.2 Threshold sensitivity and blocking (RQ2)

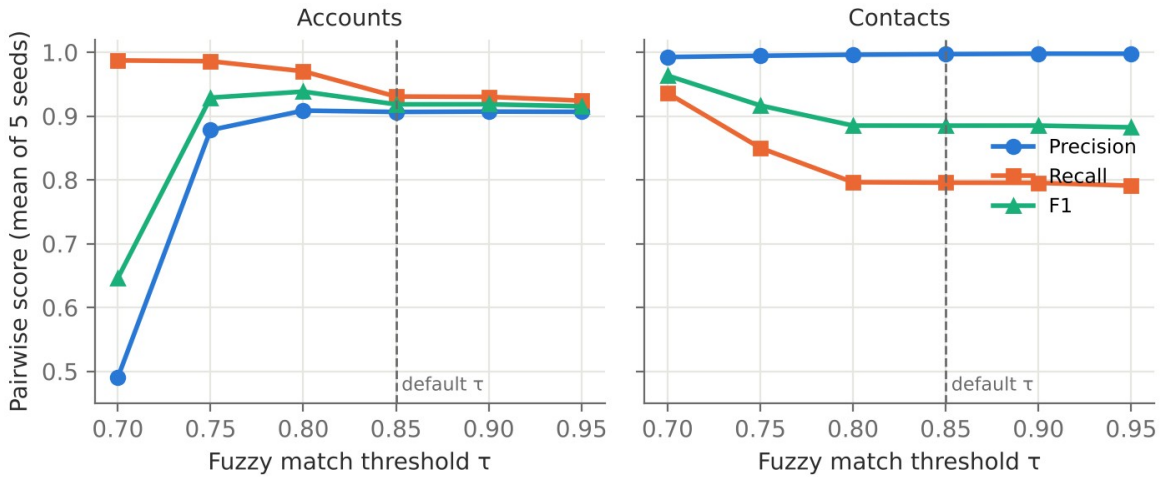


Figure 2. Precision, recall and F1 of the hybrid matcher as the fuzzy threshold τ varies (mean of 5 seeds). The dashed line marks the pre-set value used elsewhere in the evaluation.

Figure 2 shows that the two objects respond to the threshold very differently. For accounts, F1 peaked at τ = 0.80 (0.938; precision 0.909, recall 0.970) and collapsed at τ = 0.70, where precision fell to 0.490. At that threshold, distinct companies that share a leading name token (for example, several firms whose names begin with the same surname) began to link, and because clusters are connected components, a few such links chained many companies together; in the seed-7 run, 886 of 1,038 false pairs shared their first name token. For contacts, precision stayed above 0.99 across the whole range, because blocking within the golden account already excludes most false candidates, so lowering τ to 0.70 raised recall to 0.936 and F1 to 0.963. A single global threshold is therefore a poor choice: the best thresholds here differ by 0.10 between objects. These best values were selected on the evaluation data and would be optimistic if reported as expected performance; in practice τ should be calibrated per object on a labelled sample of the specific migration.

Blocking reduced the number of account pairs compared from 2,767,128 to 22,327 (99.2% fewer) at the 2,000-company scale, with F1 of 0.910 against 0.912 for exhaustive comparison: one true pair in 392 was lost. At this small scale blocking did not save wall-clock time (0.20 s against 0.19 s), because the exhaustive comparison runs as a single vectorised call, while the blocked version incurs per-block overhead. The benefit of blocking is in avoiding quadratic growth, which matters at the larger scales reported in Section 5.4.

5.3 Quality gates (RQ3)

Table 5. Detection of injected defects by the quality gates (5 seeds pooled; 319,606 source records)

Defect type	Records with defect	Flagged	Detection rate
Invalid e-mail address	1,390	1,390	100%
Reference to non-existent account	1,280	1,280	100%
Missing required name	966	961	99.5%
Negative opportunity amount	615	615	100%
Unparseable date	412	412	100%
Unmapped picklist value	366	366	100%
Revenue inflated ×1,000 (outlier rule, non-blocking)	174	174	100%
First and last names swapped	755	17	2.3%

Table 5 shows that the gates flagged every record carrying a structural defect that a rule was written for, apart from five records that also had their names swapped: the swap moved the remaining first name into the empty surname field, so the record no longer appeared incomplete. No record without an injected defect was flagged in error (0 of 313,702), and a further 1,698 clean child records were held because their parent account had been quarantined. The statistical fence caught all thousand-fold revenue errors, but swapped names, a semantic error that every field-level rule considers valid, were almost never caught; the 17 that were flagged were caught because of a second defect. Overall, 5,166 of the 5,904 records with an injected defect (87.5%) were flagged. These detection rates mainly confirm that the rules were implemented correctly, since each rule was written for a defect type that the generator injects; the instructive results are the misses.

Removing the gates showed what they prevent. Without them, an average of 1,032 ± 45 records per run (about 1.6% of the source) failed at load, broken down by object as 39 accounts, 657 contacts and 337 opportunities on average, owing to missing required fields, invalid e-mail addresses, missing parents and negative amounts. More seriously, between 39 and 55 opportunities per run were loaded with a blank stage and between 9 and 19 accounts with a blank industry, because unmapped picklist values became empty values that the target accepted. With the gates in place there were no load failures in any run.

5.4 Scaling (RQ4)

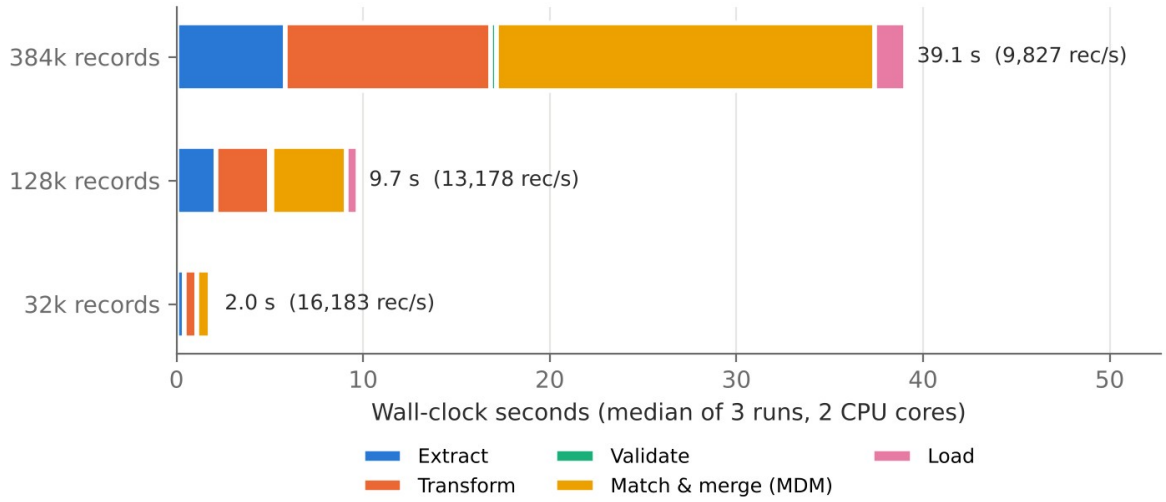


Figure 3. Wall-clock time by pipeline stage at three data volumes (median of three runs on two CPU cores; local computation only, excluding network and API latency).

Table 6. Batch pipeline results by scale

Measure	32k records	128k records	384k records
Source records	32,040	128,037	383,973
Total time (median of 3)	2.0 s	9.7 s	39.1 s
Throughput	16,183 rec/s	13,178 rec/s	9,827 rec/s
Account pairs compared	125,984	2,001,702	18,910,752
Golden accounts / contacts	4,963 / 13,319	19,847 / 53,157	59,571 / 158,894
Records unaccounted for	0	0	0
Orphaned lookups in target	0	0	0
Opportunity amount checksum	Match	Match	Match

Across all three scales, every source record was either represented in the target or recorded in the audit log, no lookup in the target pointed to a missing account, and opportunity amount totals reconciled exactly (Table 6). Throughput fell from 16,183 to 9,827 records per second as volume grew twelvefold. Figure 3 shows why: MDM grew from 36% of run time at the smallest scale to 52% at the largest, because the number of account pairs compared grew roughly 150-fold. The prefix blocks used here become large as the number of companies grows, so blocking keys need tightening, or meta-blocking techniques need applying (Papadakis et al., 2020), before the design is used at production scale. Extraction, transformation and loading grew close to linearly.

5.5 Change data capture (RQ5)

Over 120 hourly batches, 12,840 events were delivered (a mean of 107 per batch). Of these, 84 were re-deliveries and were skipped by event identifier, 12,466 were applied, 184 were rejected by the quality rules (for example, a new contact attached to an account that was in quarantine), and 106 were held by the circuit breaker. Applying a batch took a median of 13.6 ms (95th percentile 18.8 ms; maximum 23.8

ms) of local computation, so in a deployment end-to-end freshness would be governed by the hourly schedule and by API latency rather than by pipeline processing.

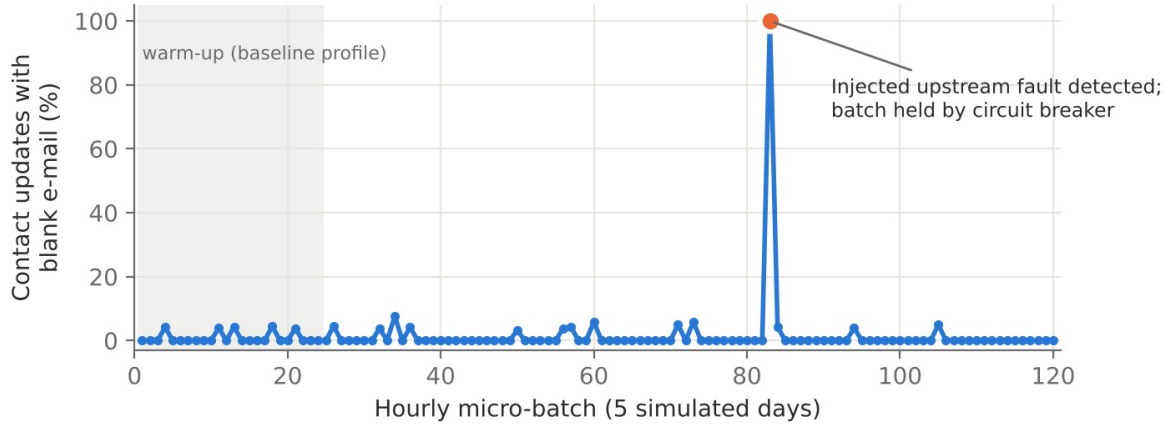


Figure 4. Share of contact updates with a blank e-mail address in each CDC batch. The first 24 batches form the baseline profile; the injected upstream fault in batch 83 was detected and held.

In normal batches the share of contact updates with a blank e-mail address averaged 0.7% and never exceeded 7.7%. In the faulty batch it was 100%, and the circuit breaker held that batch before any of it was applied (Figure 4). No other batch triggered an alarm, so there were no false alarms over 119 batches.

Table 7. Agreement between the incrementally synchronised target and a full rebuild from the final source state

Comparison	Records compared	Agreement
Opportunities: identical set of records	46,078	Yes
Opportunities: amount and stage values	46,078	100%
Contacts: golden field values per source record	57,392	99.94%
Accounts: golden field values per source record	23,750	99.36%
Contact clusters: pairwise precision against rebuild	57,392	0.994
Contact clusters: pairwise recall against rebuild	57,392	0.999

The incremental target agreed closely, though not perfectly, with a full rebuild (Table 7). Opportunities, which involve no matching, agreed exactly. The small differences for contacts and accounts arise because the incremental path matches each new record against existing clusters but never re-clusters existing records when an update changes their matching attributes. An account whose telephone number changes, for example, may join a different cluster in a rebuild. This is the divergence between incremental and batch linkage that Gruenheid et al. (2014) describe. It did not degrade accuracy against the ground truth: after CDC, contact matching F1 was 0.893 for the incremental target and 0.891 for the rebuild. A practical remedy is to run a full rebuild periodically and at cutover, which took 10.4 s at this scale, and to reconcile any differences through the crosswalk.

6. Discussion

The results support the framework's central design choices while exposing where its defaults should change. Three findings are likely to transfer beyond this simulation.

First, deterministic matching rules need the same scrutiny as probabilistic ones. In this scenario the shared-domain rule caused almost all false account merges: in the seed-7 run, 219 of the 224 false pairs produced by deterministic rules shared a web domain. In real mergers, subsidiaries, franchisees and resellers often share a domain or a main telephone number. Deterministic rules should therefore be treated as strong evidence that feeds a score, not as automatic merges, or they should be restricted to identifiers that are known to be unique, such as tax or registry numbers.

Second, matching thresholds should be calibrated per object. Contact matching benefited from a low threshold because blocking within the golden account had already removed most false candidates, whereas account matching failed abruptly below 0.75. A short labelling exercise on a sample of candidate pairs from the actual orgs, followed by a threshold sweep like that in Figure 2, is inexpensive compared with the cost of false merges, which are harder to undo after cutover than missed matches.

Third, quality gates are most valuable for what the target would not reject. Load-time constraints alone would have rejected about 1,000 records per run, which an operator would see, but they would also have accepted unmapped picklist values as blanks, which an operator would not see. Gates that refuse to default unmapped values, together with reconciliation that accounts for every source record, turn silent data loss into visible, reviewable work. The gates were nevertheless blind to semantic errors such as swapped names. Catching those needs reference data (for example, lists of given names) or learned validators (Breck et al., 2019), and should be part of the review workflow rather than assumed to be covered.

The circuit breaker illustrates a broader point about autonomy. Validating each record individually cannot detect a fault that makes many records individually legal but collectively implausible. Profiling each batch against its own history, and holding rather than applying an anomalous batch, contained the fault automatically without stopping the pipeline. "Autonomous" in this framework therefore means that routine decisions (skipping replays, quarantining defective records, holding anomalous batches) are taken without an operator, while every such decision is logged for human review. It does not mean that the pipeline makes judgement calls that cannot be audited.

For practitioners using Jitterbit, the mapping in Table 2 suggests keeping matching and survivorship in a component that can be tested in isolation, such as a database or matching service called from an operation, rather than spreading the logic across many transformations, and storing the crosswalk and audit log in a queryable staging database that outlives the migration.

7. Threats to Validity

Construct and external validity are limited chiefly by the use of synthetic data. The generator encodes assumptions about how duplicates and defects arise, and its defect rates, perturbation types and organisational overlap are plausible but not calibrated against a real merger. Real orgs also contain objects and behaviours not modelled here: cases, activities, attachments, record types, sharing rules, and automation (flows and triggers) that may fire during loading. Matching performance on real data could be higher or lower than reported.

The quality-gate detection rates are partly a consequence of design, since the rules were written for defect types that the generator injects. The informative results are the gate ablation and the misses, not the 100% rates.

Internal validity is supported by separating the ground truth from the pipeline, by fixed random seeds, and by reporting variation across five seeds. The best thresholds in Figure 2 were, however, selected on the same data used to report them, and should be treated as an upper bound.

The timing results describe a Python prototype on two CPU cores with SQLite as the target. They do not include network latency, Salesforce API limits or Jitterbit agent performance, and they should not be

read as predictions of migration duration on a production platform. The Jitterbit mapping in Table 2 is a design proposal that has not been executed in this study.

8. Conclusion and Future Work

This paper proposed a reference framework for autonomous ETL in Salesforce M&A consolidation, mapped it to Jitterbit Integration Studio, and evaluated an open reference implementation on a synthetic scenario with known ground truth. The hybrid matcher outperformed either of its components; quality gates prevented all load failures and silent loss of unmapped values; reconciliation accounted for every source record at every scale; and CDC synchronisation was idempotent under replayed events, contained an injected upstream fault without false alarms, and stayed close to a full rebuild. The evaluation also identified concrete limitations: deterministic rules that cause false merges, the need for per-object threshold calibration, blocking that grows too coarse at scale, blindness to semantic errors, and gradual divergence of incremental clusters.

Future work should (i) implement the framework on a Jitterbit Harmony deployment against Salesforce sandbox orgs to measure API-bound performance; (ii) evaluate it on a real or carefully anonymised merger dataset with manually labelled duplicate pairs; (iii) extend it to activities, cases and record types and to the effect of source automation during loading; (iv) replace fixed rules with learned matchers (Mudgal et al., 2018) and incremental re-clustering (Gruenheid et al., 2014); and (v) add reference-data checks for semantic errors.

CRediT authorship contribution statement

Imran Basha Shaik: [roles to be confirmed by the authors, e.g., Conceptualization, Writing – original draft, Writing – review & editing]. Sahla Sultana: Conceptualization, Methodology [to be confirmed]. Khader Moinuddin Shaik: Conceptualization, Methodology [to be confirmed].

Declaration of competing interest

[To be confirmed by the authors. Suggested wording: Imran Basha Shaik was previously employed by Salesforce.com. This work was not funded or reviewed by Salesforce or by Jitterbit. The authors declare no other competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.]

Funding

[To be confirmed by the authors, e.g., "This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors."]

Data availability

The data generator, pipeline implementation, experiment scripts and raw results are openly available at https://github.com/imran0547/SDFC/tree/add-autonomous-etl-salesforce-ma-consolidation/autonomous-etl-salesforce-ma-consolidation. All results reported in this article can be regenerated with the command "python run_experiments.py".

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic) in order to draft and edit the manuscript text and to help write the simulation code described in Section 4.3. After using this tool, the

authors reviewed and edited the content as needed, verified every reported result against the experiment output, checked all references, and take full responsibility for the content of the publication.

References

Alharthi, A., Krotov, V., & Bowman, M. (2017). Addressing barriers to big data. Business Horizons, 60(3), 285–292. https://doi.org/10.1016/j.bushor.2017.01.002

Batini, C., Cappiello, C., Francalanci, C., & Maurino, A. (2009). Methodologies for data quality assessment and improvement. ACM Computing Surveys, 41(3), Article 16. https://doi.org/10.1145/1541880.1541883

Breck, E., Polyzotis, N., Roy, S., Whang, S. E., & Zinkevich, M. (2019). Data validation for machine learning. Proceedings of Machine Learning and Systems, 1.

Christen, P. (2012a). Data matching: Concepts and techniques for record linkage, entity resolution, and duplicate detection. Springer. https://doi.org/10.1007/978-3-642-31164-2

Christen, P. (2012b). A survey of indexing techniques for scalable record linkage and deduplication. IEEE Transactions on Knowledge and Data Engineering, 24(9), 1537–1555. https://doi.org/10.1109/TKDE.2011.127

Christophides, V., Efthymiou, V., Palpanas, T., Papadakis, G., & Stefanidis, K. (2020). An overview of end-to-end entity resolution for big data. ACM Computing Surveys, 53(6), Article 127. https://doi.org/10.1145/3418896

Dong, X. L., & Srivastava, D. (2015). Big data integration. Morgan & Claypool. https://doi.org/10.2200/S00578ED1V01Y201404DTM040

Elmagarmid, A. K., Ipeirotis, P. G., & Verykios, V. S. (2007). Duplicate record detection: A survey. IEEE Transactions on Knowledge and Data Engineering, 19(1), 1–16. https://doi.org/10.1109/TKDE.2007.250581

Fellegi, I. P., & Sunter, A. B. (1969). A theory for record linkage. Journal of the American Statistical Association, 64(328), 1183–1210. https://doi.org/10.1080/01621459.1969.10501049

Gruenheid, A., Dong, X. L., & Srivastava, D. (2014). Incremental record linkage. Proceedings of the VLDB Endowment, 7(9), 697–708. https://doi.org/10.14778/2732939.2732943

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. MIS Quarterly, 28(1), 75–105. https://doi.org/10.2307/25148625

Inoubli, W., Aridhi, S., Mezni, H., Maddouri, M., & Mephu Nguifo, E. (2018). An experimental survey on big data frameworks. Future Generation Computer Systems, 86, 546–564. https://doi.org/10.1016/j.future.2018.04.032

Jitterbit. (2026). Salesforce connector for Jitterbit Studio [Documentation]. Retrieved November 1, 2026, from https://docs.jitterbit.com/integration-studio/design/connectors/salesforce/

Khan, Z., Anjum, A., & Kiani, S. L. (2013). Cloud based big data analytics for smart future cities. In 2013 IEEE/ACM 6th International Conference on Utility and Cloud Computing (pp. 381–386). https://doi.org/10.1109/UCC.2013.77

Kimball, R., & Caserta, J. (2004). The data warehouse ETL toolkit. Wiley.

Kleppmann, M. (2017). Designing data-intensive applications. O'Reilly Media.

Köpcke, H., & Rahm, E. (2010). Frameworks for entity matching: A comparison. Data & Knowledge Engineering, 69(2), 197–210. https://doi.org/10.1016/j.datak.2009.10.003

Loshin, D. (2009). Master data management. Morgan Kaufmann.

Mudgal, S., Li, H., Rekatsinas, T., Doan, A., Park, Y., Krishnan, G., Deep, R., Arcaute, E., & Raghavendra, V. (2018). Deep learning for entity matching: A design space exploration. In

Proceedings of the 2018 International Conference on Management of Data (pp. 19–34). https://doi.org/10.1145/3183713.3196926

Papadakis, G., Skoutas, D., Thanos, E., & Palpanas, T. (2020). Blocking and filtering techniques for entity resolution: A survey. ACM Computing Surveys, 53(2), Article 31. https://doi.org/10.1145/3377455

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. Journal of Management Information Systems, 24(3), 45–77. https://doi.org/10.2753/MIS0742-1222240302

Rahm, E., & Bernstein, P. A. (2001). A survey of approaches to automatic schema matching. The VLDB Journal, 10(4), 334–350. https://doi.org/10.1007/s007780100057

Rahm, E., & Do, H. H. (2000). Data cleaning: Problems and current approaches. IEEE Data Engineering Bulletin, 23(4), 3–13.

Roh, Y., Heo, G., & Whang, S. E. (2021). A survey on data collection for machine learning: A big data–AI integration perspective. IEEE Transactions on Knowledge and Data Engineering, 33(4), 1328–1347. https://doi.org/10.1109/TKDE.2019.2946162

Saggi, M. K., & Jain, S. (2018). A survey towards an integration of big data analytics to big insights for value-creation. Information Processing & Management, 54(5), 758–790. https://doi.org/10.1016/j.ipm.2018.01.010

Salesforce. (2026a). Change Data Capture developer guide (API version 67.0). Retrieved November 1, 2026, from https://developer.salesforce.com/docs/atlas.en-us.change_data_capture.meta/change_data_capture/cdc_what.htm

Salesforce. (2026b). Bulk API 2.0 and Bulk API developer guide (API version 67.0). Retrieved November 1, 2026, from https://developer.salesforce.com/docs/atlas.en-us.api_asynch.meta/api_asynch/bulk_api_2_0.htm

Schelter, S., Lange, D., Schmidt, P., Celikel, M., Biessmann, F., & Grafberger, A. (2018). Automating large-scale data quality verification. Proceedings of the VLDB Endowment, 11(12), 1781–1794. https://doi.org/10.14778/3229863.3229867

Simitsis, A., Vassiliadis, P., & Sellis, T. (2005). Optimizing ETL processes in data warehouses. In Proceedings of the 21st International Conference on Data Engineering (pp. 564–575). IEEE. https://doi.org/10.1109/ICDE.2005.103

Singh, D., & Reddy, C. K. (2015). A survey on platforms for big data analytics. Journal of Big Data, 2(1), Article 8. https://doi.org/10.1186/s40537-014-0008-6

Vassiliadis, P. (2009). A survey of extract–transform–load technology. International Journal of Data Warehousing and Mining, 5(3), 1–27. https://doi.org/10.4018/jdwm.2009070101

Wang, R. Y., & Strong, D. M. (1996). Beyond accuracy: What data quality means to data consumers. Journal of Management Information Systems, 12(4), 5–33. https://doi.org/10.1080/07421222.1996.11518099

Winkler, W. E. (1990). String comparator metrics and enhanced decision rules in the Fellegi-Sunter model of record linkage. In Proceedings of the Section on Survey Research Methods (pp. 354–359). American Statistical Association.